

Universidad ORT Uruguay

Facultad de Ingeniería

Evidencia de la ejecución de las pruebas de la API con Postman

Obligatorio 1 – Diseño de Aplicaciones 2: FarmeORT

Juan Ferreiro – 328344

Sebastián Luciardi – 274754

Andrés Bustamante – 302420

Grupo M6C – Tutores: Alexander Wieler y Franco Bonavita

2026

Repositorio: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420

Declaración de autoría

Nosotros, Juan Ferreiro, Sebastián Luciardi y Andrés Bustamante, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el Obligatorio 1 de Diseño de Aplicaciones 2;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.

Juan Ferreiro, Sebastián Luciardi y Andrés Bustamante – 08/10/2026

Abstract

Este documento muestra qué pruebas de la API de FarmeORT se ejecutaron con Postman y qué resultado dieron. Para las cinco funcionalidades con asterisco de la letra (inscripción a un torneo, lanzar una skill, fallar una batalla, catálogo de skills y generar rondas y finalizar el torneo) se probaron valores inválidos y límite, tipos erróneos, datos vacíos y nulos, campos omitidos y redundantes, body vacío, mensajes con formato inválido, autenticación, permisos y cada regla de negocio. Del resto de los endpoints se muestra el caso feliz. La colección de evidencia tiene 232 requests y 510 assertions, y todas pasan contra la API local. El documento explica cómo está organizada la colección, cómo se ejecuta y por qué se eligió cada caso, e incluye el enlace al video con la ejecución.

Índice

1. Qué se probó y cómo 4

1.1 Dos colecciones en la carpeta Datos 4

1.2 Organización de la colección 4

1.3 Cómo se ejecuta 5

2. Resultado de la ejecución 5

3. Casos de las funcionalidades con asterisco 6

3.1 Catálogo de skills (2.2) 7

3.2 Inscripción a un torneo (1.5) 9

3.3 Lanzar una skill (1.8) 10

3.4 Fallar una batalla (1.9) 12

3.5 Generar rondas y finalizar el torneo (2.5) 13

4. Caso feliz del resto de los endpoints 14

5. Hallazgos durante las pruebas 15

6. Uso de IA 16

7. Referencias bibliográficas 17

1. Qué se probó y cómo

La letra del obligatorio [1] pide una colección de Postman con pruebas para todas las funcionalidades y, para las marcadas con asterisco, casos elaborados: valores inválidos y límite, tipos de datos erróneos, datos vacíos y nulos, omisión de campos obligatorios, campos redundantes, body vacío, formato de los mensajes inválido y las reglas de negocio, como el alta de elementos existentes o la baja de inexistentes. Este documento muestra qué prueba se ejecutó para cada caso y qué resultado dio.

1.1 Dos colecciones en la carpeta Datos

En la carpeta `Datos` del repositorio [2] hay dos colecciones (decisión D-79 de la bitácora). `FarmeORT-Evidencia.postman_collection.json` es la que se muestra en este documento y en el video: tiene todos los casos de las funcionalidades con asterisco y, del resto, el caso feliz de cada endpoint. `FarmeORT.postman_collection.json` es la colección completa, con 422 requests que cubren además los errores y bordes de los demás endpoints, y se usa como regresión.

Se separaron porque la letra limita el video a 15 minutos y este documento a 15 páginas. Con la colección completa, los casos de usuarios, torneos o repertorio ocupan el mismo espacio que los de las funcionalidades con asterisco, que son las que la letra evalúa especialmente. La colección de evidencia se derivó de la completa con un script, así que sus requests y sus tests son los mismos que ya estaban probados.

1.2 Organización de la colección

La colección tiene una carpeta por recurso y, dentro, una subcarpeta por endpoint con su método y ruta (por ejemplo, `POST /inscripciones`), como en la especificación de la API (D-78). Cuando un recurso necesita datos de otros, tiene al principio una subcarpeta Preparación que los crea por la API: organizadores, jugadores, skills, torneos e inscripciones. Cada request se llama como su caso, con el status esperado al final (por ejemplo, «Inscripciones - cupo de aceptadas completo → 409»), para que el Runner y el video se lean sin abrir cada request.

Cada request tiene tests que se ejecutan con su respuesta:

- **Status:** el status esperado, en todos los requests;
- **Código de error:** el código de error propio (`errorCode`), en todos los casos de error, porque dos errores con el mismo status pueden tener causas distintas (por ejemplo, un 409 por torneo no abierto y otro por cupo completo);
- **Formato:** el formato del cuerpo de error (`userMessage`, `errorCode`, `moreInfo` y `detalles`) y el campo que falló en las validaciones;
- **Efectos:** el header `Location` en las altas y los efectos en otros recursos: que la inscripción del perdedor quede Eliminada, que el ELO cambie con los valores previos o que el torneo pase a En curso.

1.3 Cómo se ejecuta

La API se levanta con el perfil http de la WebAPI (puerto 5294) sobre SQL Server, con la base creada por las migraciones o por el script `Datos/FarmeORT-BaseVacia.sql`. En un environment de Postman se definen `baseUrl` (`http://localhost:5294`) y `adminContrasena`, la contraseña del administrador precargado, que no se guarda en la colección. Después se ejecuta la colección completa con el Collection Runner, en orden.

La colección no depende de datos cargados a mano. Los emails, nicknames y códigos se generan en cada corrida con un sufijo único, y los tokens e ids se guardan en variables de colección a medida que se crean. Por eso se puede correr varias veces sobre la misma base sin que choquen las restricciones de unicidad. El orden de las carpetas es el orden de ejecución y no hay saltos entre carpetas; la última, Cierre, cierra la sesión del administrador.

2. Resultado de la ejecución

La colección se ejecutó completa contra la API local el 08/10/2026. Todos los requests respondieron lo esperado y no falló ningún test.

Tabla 1. Resumen de la ejecución de la colección de evidencia

Carpeta	Requests
Jugadores	1
Sesiones	3

Carpeta	Requests
Usuarios	5
Skills	50
Torneos	7
Repertorio	7
Skills obligatorias	8
Inscripciones	40
Rondas	35
Batallas	2
Lanzamientos	29
Fallos	27
Rondas siguientes	7
Finalización	9
Ranking	1
Cierre	1
Total (261 ejecuciones por los requests que se repiten)	232

Los 232 requests suman 510 assertions. Hay 261 ejecuciones porque algunos pasos de preparación se repiten para cada jugador (registro, login, declaración de skills e inscripción). En el Collection Runner de Postman la colección completa dio 510 tests aprobados, 0 fallidos y 0 errores, en 21,9 segundos (Ilustración 1).

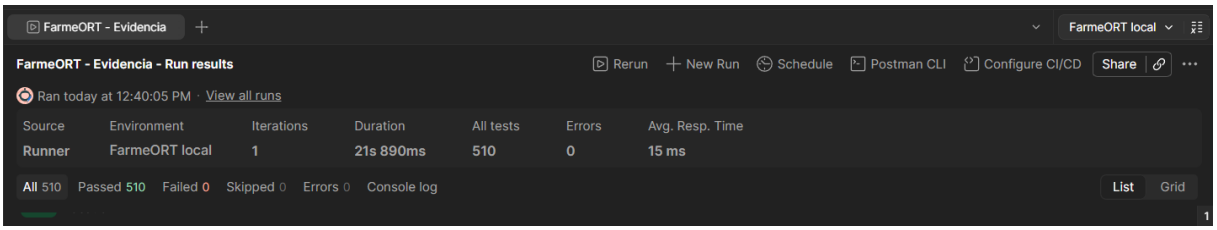


Ilustración 1. Resultado del Collection Runner con la colección de evidencia

[COMPLETAR] Enlace al video de YouTube con la ejecución (público, hasta 15 minutos).

3. Casos de las funcionalidades con asterisco

La Tabla 2 resume qué categorías de la letra se probaron en cada funcionalidad. Las secciones siguientes explican por qué se eligieron los casos y listan cada uno con su request, el resultado esperado y el obtenido.

Tabla 2. Categorías de la letra cubiertas por funcionalidad

Categoría	2.2 Catálogo	1.5 Inscripción	1.8 Lanzar	1.9 Fallar	2.5 Rondas y finalizar
Valores inválidos	Sí	Sí	Sí	Sí	Sí
Valores límite	Largos 5/20, 3/40, 20/300 y dificultad 1/5	Id 0 y -1	Id 0 y -1	Comentario 9, 10, 500 y 501	Cupo incompleto y completo
Tipo de dato erróneo	Sí	Sí	Sí	Sí	No aplica (sin body)
Datos vacíos y nulos	Sí	Sí	Sí	Sí	No aplica
Campos obligatorios omitidos	Sí	Sí	Sí	Sí	No aplica
Campos redundantes	Sí	Sí	Sí	Sí	No aplica
Body vacío {} y JSON mal formado	Sí	Sí	Sí	Sí	No aplica
Body que no es JSON (415)	Sí	Sí	Sí	Sí	No aplica
Sin token (401) y rol incorrecto (403)	Sí	Sí	Sí	Sí	Sí
Recurso inexistente (404)	Sí	Sí	Sí	Sí	Sí
Reglas de negocio (409)	Sí	Sí	Sí	Sí	Sí

Generar rondas y finalizar el torneo no reciben body: el torneo sale de la ruta y quien lo pide, del token. Por eso los casos de body no aplican, y la evidencia de esa funcionalidad está en los permisos y en las reglas de estado.

3.1 Catálogo de skills (2.2)

El catálogo tiene alta, modificación, consulta con filtros y cambio de estado. Los límites se prueban justo en el valor permitido y uno afuera: código de 5 y 20 caracteres frente a 4 y 21, nombre de 3 y 40 frente a 2 y 41, descripción de 20 y 300 frente a 19 y 301, y dificultad 1 y 5 frente a 0 y 6. Así un error de «menor» por «menor o igual» se detecta. El código único se prueba también con otras mayúsculas, porque la regla es de negocio y no de la base: «SIXSEVEN» y «sixseven» son el mismo código.

Los filtros fuera de rango no se rechazan (una categoría inexistente o un `limit` mayor al tope), como decidió el equipo para no trancar al usuario; un tipo mal formado sí es 400. La skill inactiva se prueba donde la regla tiene efecto: no se puede declarar en un repertorio ni asociar como obligatoria (409 `SKILL_INACTIVE`), y como no es retroactiva, una skill desactivada que ya estaba declarada se lanza en batalla (turnos 3 y 4 de la Tabla 5).

Tabla 3. Casos del catálogo de skills

#	Caso	Request	Esperado	Resultado
1	Alta válida	POST /skills	201	Aprobado

#	Caso	Request	Esperado	Resultado
2	Alta con valores mínimos	POST /skills	201	Aprobado
3	Alta con valores máximos	POST /skills	201	Aprobado
4	Campos redundantes se ignoran	POST /skills	201	Aprobado
5	Alta con código existente	POST /skills	409 SKILL_CODE_ALREADY_EXISTS	Aprobado
6	Alta con código existente en otras mayúsculas	POST /skills	409 SKILL_CODE_ALREADY_EXISTS	Aprobado
7	Body vacío {}	POST /skills	400 VALIDATION_ERROR	Aprobado
8	JSON mal formado	POST /skills	400 VALIDATION_ERROR	Aprobado
9	Body que no es JSON	POST /skills	415 UNSUPPORTED_MEDIA_TYPE	Aprobado
10	Campos nulos	POST /skills	400 VALIDATION_ERROR	Aprobado
11	Campos obligatorios omitidos	POST /skills	400 VALIDATION_ERROR	Aprobado
12	Código con guion	POST /skills	400 VALIDATION_ERROR	Aprobado
13	Código de 4 caracteres	POST /skills	400 VALIDATION_ERROR	Aprobado
14	Código de 21 caracteres	POST /skills	400 VALIDATION_ERROR	Aprobado
15	Nombre de 2 caracteres	POST /skills	400 VALIDATION_ERROR	Aprobado
16	Nombre de 41 caracteres	POST /skills	400 VALIDATION_ERROR	Aprobado
17	Descripción de 19 caracteres	POST /skills	400 VALIDATION_ERROR	Aprobado
18	Descripción de 301 caracteres	POST /skills	400 VALIDATION_ERROR	Aprobado
19	Categoría inexistente	POST /skills	400 VALIDATION_ERROR	Aprobado
20	Dificultad 0	POST /skills	400 VALIDATION_ERROR	Aprobado
21	Dificultad 6	POST /skills	400 VALIDATION_ERROR	Aprobado
22	Dificultad de tipo incorrecto	POST /skills	400 VALIDATION_ERROR	Aprobado
23	Alta sin token	POST /skills	401 INVALID_TOKEN	Aprobado
24	Alta con token inválido	POST /skills	401 INVALID_TOKEN	Aprobado
25	Jugador no puede crear skills	POST /skills	403 FORBIDDEN	Aprobado
26	Jugador lista el catálogo	GET /skills	200	Aprobado
27	Filtra por nombre, categoría y estado	GET /skills?nombre=…&categoria=Gestual&activa=true	200	Aprobado
28	Pagina el listado	GET /skills?limit=1&offset=0	200	Aprobado
29	Limit mayor al tope usa el tope	GET /skills?limit=500	200	Aprobado
30	Offset negativo empieza desde el principio	GET /skills?nombre=…&offset=-3	200	Aprobado
31	Categoría inexistente no filtra	GET /skills?nombre=…&categoria=Cocina	200	Aprobado
32	Dificultad fuera de rango no filtra	GET /skills?nombre=…&dificultad=9	200	Aprobado
33	Consulta sin token	GET /skills	401 INVALID_TOKEN	Aprobado
34	Jugador consulta una skill	GET /skills/{skillId}	200	Aprobado
35	Consulta de skill inexistente	GET /skills/999999	404 SKILL_NOT_FOUND	Aprobado
36	Modificación válida	PUT /skills/{skillId}	200	Aprobado
37	Modificación que solo cambia mayúsculas del código	PUT /skills/{skillId}	200	Aprobado
38	Modificación de skill inexistente	PUT /skills/999999	404 SKILL_NOT_FOUND	Aprobado
39	Modificación con el código de otra skill	PUT /skills/{skillId}	409 SKILL_CODE_ALREADY_EXISTS	Aprobado
40	Modificación con datos inválidos	PUT /skills/{skillId}	400 VALIDATION_ERROR	Aprobado
41	Jugador no puede modificar skills	PUT /skills/{skillId}	403 FORBIDDEN	Aprobado

#	Caso	Request	Esperado	Resultado
42	Desactivar	PATCH /skills/{skillId}	200	Aprobado
43	Filtra las inactivas	GET /skills?nombre=…&activa=false	200	Aprobado
44	Reactivar	PATCH /skills/{skillId}	200	Aprobado
45	Cambio de estado sin activa	PATCH /skills/{skillId}	400 VALIDATION_ERROR	Aprobado
46	Cambio de estado con tipo incorrecto	PATCH /skills/{skillId}	400 VALIDATION_ERROR	Aprobado
47	Cambio de estado de skill inexistente	PATCH /skills/999999	404 SKILL_NOT_FOUND	Aprobado
48	Jugador no puede cambiar el estado	PATCH /skills/{skillId}	403 FORBIDDEN	Aprobado
49	Administrador desactiva la segunda skill	PATCH /skills/{skillId}	200	Aprobado
50	Declarar una skill inactiva	POST /repertorios	409 SKILL_INACTIVE	Aprobado
51	Administrador reactiva la segunda skill	PATCH /skills/{skillId}	200	Aprobado
52	Administrador desactiva la tercera skill	PATCH /skills/{skillId}	200	Aprobado
53	Asociar una skill inactiva	POST /torneos/{id}/skills-obligatorias	409 SKILL_INACTIVE	Aprobado

3.2 Inscripción a un torneo (1.5)

Cada motivo de rechazo de la letra tiene su caso, con su código propio: torneo no abierto (cancelado y en curso), inscripción aceptada previa, skills obligatorias faltantes y cupo completo. El cupo se prueba con un torneo de cupo 4: se inscriben cuatro jugadores y el quinto recibe 409 `TOURNAMENT_FULL`. El caso del torneo en curso lo intenta un jugador que tiene las dos obligatorias, para que el único motivo del rechazo sea el estado. También se prueba la cancelación: la propia, una ya cancelada, una ajena, una inexistente y con el torneo ya en curso.

En el body solo va el torneo. El caso de campos redundantes manda además `jugadorId` de otro jugador, `id` y `estado`, y verifica que la inscripción queda a nombre del jugador del token y Aceptada: la identidad sale siempre del token. Un `torneoId` 0 o negativo responde 404, como define la especificación (R09): el DTO no exige un rango y el id se trata como un torneo que no existe.

Tabla 4. Casos de la inscripción a un torneo

#	Caso	Request	Esperado	Resultado
1	Jugador se inscribe en el torneo	POST /inscripciones	201	Aprobado
2	El torneo muestra la cantidad de aceptadas	GET /torneos/{id}	200	Aprobado
3	El detalle del torneo muestra obligatorias y jugadores aceptados	GET /torneos/{id}	200	Aprobado
4	Ya tiene una inscripción aceptada	POST /inscripciones	409 ALREADY_REGISTERED	Aprobado
5	Le faltan skills obligatorias	POST /inscripciones	409 MISSING_MANDATORY_SKILLS	Aprobado
6	Torneo cancelado	POST /inscripciones	409 TOURNAMENT_NOT_OPEN	Aprobado
7	Torneo inexistente	POST /inscripciones	404 TOURNAMENT_NOT_FOUND	Aprobado

#	Caso	Request	Esperado	Resultado
8	Torneo 0	POST /inscripciones	404 TOURNAMENT_NOT_FOUND	Aprobado
9	Torneo -1	POST /inscripciones	404 TOURNAMENT_NOT_FOUND	Aprobado
10	Body vacío {}	POST /inscripciones	400 VALIDATION_ERROR	Aprobado
11	Torneo null	POST /inscripciones	400 VALIDATION_ERROR	Aprobado
12	Torneo de tipo incorrecto	POST /inscripciones	400 VALIDATION_ERROR	Aprobado
13	JSON mal formado	POST /inscripciones	400 VALIDATION_ERROR	Aprobado
14	Body que no es JSON	POST /inscripciones	415 UNSUPPORTED_MEDIA_TYPE	Aprobado
15	Inscripción sin token	POST /inscripciones	401 INVALID_TOKEN	Aprobado
16	Organizador no puede inscribirse	POST /inscripciones	403 FORBIDDEN	Aprobado
17	Inscribe al jugador 1 del cupo	POST /inscripciones	201	Aprobado
18	Inscribe al jugador 2 del cupo	POST /inscripciones	201	Aprobado
19	Inscribe al jugador 3 del cupo	POST /inscripciones	201	Aprobado
20	Inscribe al jugador 4 del cupo	POST /inscripciones	201	Aprobado
21	Cupo de aceptadas completo	POST /inscripciones	409 TOURNAMENT_FULL	Aprobado
22	Jugador cancela su inscripción	POST /inscripciones/{id}/cancelaciones	200	Aprobado
23	Vuelve a inscribirse; los campos redundantes se ignoran	POST /inscripciones	201	Aprobado
24	Cancelar una inscripción ya cancelada	POST /inscripciones/{id}/cancelaciones	409 REGISTRATION_NOT_ACTIVE	Aprobado
25	Cancelar una inscripción ajena	POST /inscripciones/{id}/cancelaciones	403 NOT_REGISTRATION_OWNER	Aprobado
26	Cancelar sin token	POST /inscripciones/{id}/cancelaciones	401 INVALID_TOKEN	Aprobado
27	Cancelar una inscripción inexistente	POST /inscripciones/999999/cancelaciones	404 REGISTRATION_NOT_FOUND	Aprobado
28	Con el torneo en curso no se admiten inscripciones	POST /inscripciones	409 TOURNAMENT_NOT_OPEN	Aprobado
29	Con el torneo en curso no se cancela una inscripción	POST /inscripciones/{id}/cancelaciones	409 TOURNAMENT_NOT_OPEN	Aprobado

3.3 Lanzar una skill (1.8)

La batalla de prueba se arma en la carpeta Rondas: un torneo de cupo 4 con dos skills obligatorias, cuatro jugadores con cinco skills declaradas y una skill que se desactiva después de declararla. Los casos siguen la batalla turno por turno, así que cada regla se prueba en el momento en que se puede violar: el turno del rival antes del primer lanzamiento, la skill repetida en el turno 3, las obligatorias pendientes cuando a un jugador le quedan tantas como turnos, y la batalla en arbitraje después del octavo lanzamiento. Los casos de body se ejecutan antes del

primer turno, para que el rechazo no dependa del estado.

Tabla 5. Casos de lanzar una skill

#	Caso	Request	Esperado	Resultado
1	Sin token	POST /batallas/{id}/lanzamientos	401 INVALID_TOKEN	Aprobado
2	Rol organizador	POST /batallas/{id}/lanzamientos	403 FORBIDDEN	Aprobado
3	Jugador ajeno a la batalla	POST /batallas/{id}/lanzamientos	409 PLAYER_NOT_IN_BATTLE	Aprobado
4	Turno del rival	POST /batallas/{id}/lanzamientos	409 NOT_PLAYER_TURN	Aprobado
5	Skill no declarada	POST /batallas/{id}/lanzamientos	409 SKILL_NOT_IN_REPERTOIRE	Aprobado
6	Skill inexistente	POST /batallas/{id}/lanzamientos	404 SKILL_NOT_FOUND	Aprobado
7	Batalla inexistente	POST /batallas/2147483647/lanzamientos	404 BATTLE_NOT_FOUND	Aprobado
8	Body ausente	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
9	Body null	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
10	Skill omitida	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
11	Skill null	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
12	Skill cero	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
13	Skill negativa	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
14	Tipo incorrecto	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
15	JSON mal formado	POST /batallas/{id}/lanzamientos	400 VALIDATION_ERROR	Aprobado
16	Body que no es JSON	POST /batallas/{id}/lanzamientos	415 UNSUPPORTED_MEDIA_TYPE	Aprobado
17	Turno 1	POST /batallas/{id}/lanzamientos	201	Aprobado
18	Turno 2; los campos redundantes se ignoran	POST /batallas/{id}/lanzamientos	201	Aprobado
19	Repetición del mismo jugador	POST /batallas/{id}/lanzamientos	409 SKILL_ALREADY_THROWN	Aprobado
20	Turno 3 skill desactivada declarada	POST /batallas/{id}/lanzamientos	201	Aprobado
21	Turno 4 skill desactivada declarada	POST /batallas/{id}/lanzamientos	201	Aprobado
22	Obligatorias pendientes	POST /batallas/{id}/lanzamientos	409 MANDATORY_SKILL_PENDING	Aprobado
23	Turno 5	POST /batallas/{id}/lanzamientos	201	Aprobado
24	Turno 6	POST /batallas/{id}/lanzamientos	201	Aprobado
25	Turno 7	POST /batallas/{id}/lanzamientos	201	Aprobado

#	Caso	Request	Esperado	Resultado
26	Turno 8	POST /batallas/{id}/lanzamientos	201	Aprobado
27	Batalla en arbitraje	POST /batallas/{id}/lanzamientos	409 BATTLE_NOT_PENDING	Aprobado
28	Historial completo	GET /batallas/{id}/lanzamientos	200	Aprobado

3.4 Fallar una batalla (1.9)

Se fallan las tres situaciones de la letra: una batalla en arbitraje (después de los ocho lanzamientos), una pendiente, que resuelve un abandono, y la final del torneo. El comentario se prueba en sus cuatro bordes (9, 10, 500 y 501 caracteres) y también vacío y en blanco. El fallo exitoso verifica los efectos que pide la letra: la batalla queda Finalizada con su ganador, el ELO de los dos cambia según los valores previos de ambos, y la inscripción del perdedor pasa a Eliminada. Solo puede fallarla el organizador asignado al torneo, así que se prueban también un jugador, un administrador y otro organizador.

Tabla 6. Casos de fallar una batalla

#	Caso	Request	Esperado	Resultado
1	Sin token	POST /batallas/{id}/fallos	401 INVALID_TOKEN	Aprobado
2	Rol jugador	POST /batallas/{id}/fallos	403 FORBIDDEN	Aprobado
3	Rol administrador	POST /batallas/{id}/fallos	403 FORBIDDEN	Aprobado
4	Organizador ajeno	POST /batallas/{id}/fallos	403 NOT_TOURNAMENT_ORGANIZER	Aprobado
5	Batalla inexistente	POST /batallas/2147483647/fallos	404 BATTLE_NOT_FOUND	Aprobado
6	Ganador ajeno	POST /batallas/{id}/fallos	409 PLAYER_NOT_IN_BATTLE	Aprobado
7	Body ausente	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
8	Body null	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
9	Campos omitidos	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
10	Ganador omitido	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
11	Ganador null	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
12	Ganador cero	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
13	Ganador negativo	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
14	Tipo incorrecto	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
15	Comentario omitido	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
16	Comentario null	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
17	Comentario vacío	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
18	Comentario blanco	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
19	Comentario de 9	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
20	Comentario de 501	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
21	JSON mal formado	POST /batallas/{id}/fallos	400 VALIDATION_ERROR	Aprobado
22	Body que no es JSON	POST /batallas/{id}/fallos	415 UNSUPPORTED_MEDIA_TYPE	Aprobado

#	Caso	Request	Esperado	Resultado
23	Arbitraje comentario de 10	POST /batallas/{id}/fallos	201	Aprobado
24	Inscripción del perdedor eliminada	GET /inscripciones?torneoId=…&limit=100	200	Aprobado
25	Segundo fallo	POST /batallas/{id}/fallos	409 BATTLE_NOT_JUDGEABLE	Aprobado
26	Abandono comentario de 500	POST /batallas/{id}/fallos	201	Aprobado
27	Listado de finalizadas	GET /batallas?torneoId=…&estado=Finalizada	200	Aprobado
28	El organizador falla la final; los campos redundantes se ignoran	POST /batallas/{id}/fallos	201	Aprobado

3.5 Generar rondas y finalizar el torneo (2.5)

La primera ronda se intenta con el cupo incompleto (409 `TOURNAMENT_NOT_FULL`) y se genera cuando se inscribe el cuarto jugador. El test de la generación verifica el emparejamiento documentado: con el mismo ELO, el desempate es por id y el primero enfrenta al último. Después se prueba que no se genera otra ronda con batallas sin resolver, que con la primera resuelta se genera la final, y que con un solo jugador sin eliminar corresponde finalizar y no generar. La finalización se rechaza con dos jugadores vivos, con la final sin resolver, en un torneo cancelado y dos veces seguidas.

Tabla 7. Casos de generar rondas y finalizar el torneo

#	Caso	Request	Esperado	Resultado
1	Generar sin token	POST /torneos/{id}/rondas	401 INVALID_TOKEN	Aprobado
2	Un jugador no puede generar	POST /torneos/{id}/rondas	403 FORBIDDEN	Aprobado
3	Organizador que no es el asignado	POST /torneos/{id}/rondas	403 NOT_TOURNAMENT_ORGANIZER	Aprobado
4	Generar en un torneo inexistente	POST /torneos/999999/rondas	404 TOURNAMENT_NOT_FOUND	Aprobado
5	Con el cupo incompleto no se genera	POST /torneos/{id}/rondas	409 TOURNAMENT_NOT_FULL	Aprobado
6	Torneo cancelado, aunque lo pida un administrador	POST /torneos/{id}/rondas	409 TOURNAMENT_NOT_OPEN	Aprobado
7	Antes de generarla el listado está vacío	GET /torneos/{id}/rondas	200	Aprobado
8	El organizador genera la primera ronda	POST /torneos/{id}/rondas	201	Aprobado
9	El detalle del torneo trae el cuadro	GET /torneos/{id}	200	Aprobado
10	No se genera otra con la primera sin resolver	POST /torneos/{id}/rondas	409 ROUND_NOT_RESOLVED	Aprobado
11	Con dos jugadores sin eliminar no hay campeón	POST /torneos/{id}/finalizaciones	409 TOURNAMENT_WITHOUT_CHAMPION	Aprobado
12	Con la primera resuelta se genera la final	POST /torneos/{id}/rondas	201	Aprobado
13	El cuadro muestra las dos rondas	GET /torneos/{id}	200	Aprobado
14	No se genera otra con la final sin resolver	POST /torneos/{id}/rondas	409 ROUND_NOT_RESOLVED	Aprobado

#	Caso	Request	Esperado	Resultado
15	Con la final sin resolver no se finaliza	POST /torneos/{id}/finalizaciones	409 TOURNAMENT_HAS_UNRESOLVED_BATTLES	Aprobado
16	Con un solo jugador sin eliminar corresponde finalizar	POST /torneos/{id}/rondas	409 TOURNAMENT_READY_TO_FINISH	Aprobado
17	Finalizar sin token	POST /torneos/{id}/finalizaciones	401 INVALID_TOKEN	Aprobado
18	Un jugador no puede finalizar	POST /torneos/{id}/finalizaciones	403 FORBIDDEN	Aprobado
19	Organizador que no es el asignado	POST /torneos/{id}/finalizaciones	403 NOT_TOURNAMENT_ORGANIZER	Aprobado
20	Torneo inexistente	POST /torneos/999999/finalizaciones	404 TOURNAMENT_NOT_FOUND	Aprobado
21	Un torneo cancelado no está en curso	POST /torneos/{id}/finalizaciones	409 TOURNAMENT_NOT_IN_PROGRESS	Aprobado
22	El organizador finaliza el torneo	POST /torneos/{id}/finalizaciones	200	Aprobado
23	No se finaliza dos veces	POST /torneos/{id}/finalizaciones	409 TOURNAMENT_NOT_IN_PROGRESS	Aprobado
24	Un torneo finalizado no genera rondas	POST /torneos/{id}/rondas	409 TOURNAMENT_NOT_OPEN	Aprobado
25	El detalle muestra el torneo finalizado con su cuadro	GET /torneos/{id}	200	Aprobado

4. Caso feliz del resto de los endpoints

Del resto de los endpoints se muestra el caso feliz, que comprueba que el endpoint existe y responde según la especificación. Sus casos de error y de borde están en la colección completa. La Tabla 8 no incluye los requests de preparación, que también son casos felices de estos endpoints.

Tabla 8. Caso feliz de los demás endpoints

#	Caso	Request	Esperado	Resultado
1	Registro público	POST /jugadores	201	Aprobado
2	Login del jugador registrado	POST /sesiones	201	Aprobado
3	Login administrador	POST /sesiones	201	Aprobado
4	Logout del jugador	DELETE /sesiones/{token}	204	Aprobado
5	Alta válida conserva rol administrativo elegido	POST /usuarios	201	Aprobado
6	Administrador lista los usuarios	GET /usuarios	200	Aprobado
7	Administrador consulta Location del jugador	GET /usuarios/{id}	200	Aprobado
8	Modificación válida reemplaza datos y rol	PUT /usuarios/{id}	200	Aprobado
9	Eliminación válida	DELETE /usuarios/{id}	204	Aprobado

#	Caso	Request	Esperado	Resultado
10	Alta válida	POST /torneos	201	Aprobado
11	Alta con cupo 4 e inicio hoy	POST /torneos	201	Aprobado
12	Jugador filtra por nombre y estado	GET /torneos?nombre=…&estado=Abierto	200	Aprobado
13	Anónimo consulta el detalle	GET /torneos/{id}	200	Aprobado
14	Modificación válida	PUT /torneos/{id}	200	Aprobado
15	Cancelación	POST /torneos/{id}/cancelaciones	200	Aprobado
16	Jugador declara una skill	POST /repertorios	201	Aprobado
17	Declara otra skill con nivel 5	POST /repertorios	201	Aprobado
18	Jugador consulta su repertorio	GET /repertorios	200	Aprobado
19	Quita una skill	DELETE /repertorios/{skillId}	204	Aprobado
20	Organizador asocia una skill	POST /torneos/{id}/skills-obligatorias	201	Aprobado
21	Asocia una segunda skill	POST /torneos/{id}/skills-obligatorias	201	Aprobado
22	Anónimo consulta las obligatorias	GET /torneos/{id}/skills-obligatorias	200	Aprobado
23	Organizador quita una obligatoria	DELETE /torneos/{id}/skills-obligatorias/{skillId}	204	Aprobado
24	Jugador lista las suyas filtrando por torneo y estado	GET /inscripciones?torneoId=…&estado=Aceptada	200	Aprobado
25	Jugador consulta su inscripción	GET /inscripciones/{id}	200	Aprobado
26	El anónimo lista las rondas	GET /torneos/{id}/rondas	200	Aprobado
27	Consulta de la ronda 1	GET /torneos/{id}/rondas/1	200	Aprobado
28	Listado organizador	GET /batallas?torneoId=…	200	Aprobado
29	Detalle inicial y turno	GET /batallas/{id}	200	Aprobado
30	Historial vacío	GET /batallas/{id}/lanzamientos	200	Aprobado
31	Anónimo con la cantidad por defecto	GET /rankings	200	Aprobado
32	Logout administrador	DELETE /sesiones/{token}	204	Aprobado

5. Hallazgos durante las pruebas

Al completar la colección se encontraron dos problemas, y los dos estaban en las pruebas, no en la API.

Una prueba que dependía de los datos de la base. La prueba del ranking filtrado por torneo comparaba las batallas de cada participante con el ranking general, que tiene un tope de 100 jugadores. En una base con corridas anteriores, un jugador eliminado con ELO 1184 quedaba fuera del top 100 por el desempate por id, y la prueba fallaba aunque la API respondía bien. Se cambió para comparar con las batallas Finalizadas del torneo.

Dos criterios para el id 0. Un `torneoId` 0 responde 404 en la inscripción y 400 en lanzamientos y fallos, donde el DTO exige un id positivo. No es un error: la especificación define los dos comportamientos (R09 y reglas generales). Se agregaron los casos 0 y -1 para que el comportamiento quede probado.

6. Uso de IA

Este documento lo redactó Claude Code (modelo Claude Opus 5.5) a partir de la colección, la letra, la especificación de la API y la bitácora. También generó las tablas de casos leyendo la colección, para que coincidan con lo que se ejecuta. Claude Code revisó la colección contra las categorías de la letra, propuso los casos faltantes (body nulo y mal formado en la inscripción, campos redundantes, el 415, el torneo en curso) y armó la colección de evidencia. El integrante definió el alcance: una colección aparte para la evidencia, dejar el 404 para el id 0, y qué funcionalidades llevan solo el caso feliz. El integrante también encontró, al correr la colección, el fallo de la prueba del ranking.

Reflexión del equipo. La IA nos sirvió sobre todo para revisar la colección contra la letra y para que el documento coincida con lo que se ejecuta. Cruzar 400 requests con la lista de categorías de la letra a mano es lento y es fácil saltearse algo; así aparecieron los casos que faltaban en la inscripción, los campos redundantes y el 415. Las tablas de este documento se generaron leyendo la colección, y por eso no hay casos escritos que no existan en Postman. También aprendimos a no aceptar sus conclusiones sin contrastarlas: la IA presentó el 404 del torneo 0 como una inconsistencia, y al revisar la especificación vimos que era el comportamiento definido. La primera clasificación automática de los requests dejaba bloques a medio armar, y el runner que escribió para verificar las colecciones dio un fallo falso hasta que se corrigió. La prueba del ranking que fallaba la encontramos nosotros al correr la colección, y una corrida interrumpida en Postman nos recordó que los números se toman de una ejecución completa.

Sobre las pruebas de la API, lo que más aprendimos es que la independencia de los tests (la I de FIRST) también vale en Postman. Una prueba que compara contra datos que dejaron corridas anteriores puede pasar en una base limpia y fallar en la nuestra sin que la API haya cambiado. Por eso cada corrida genera sus propios emails, códigos y tokens, y los casos verifican el errorCode y no solo el status: dos 409 con causas distintas tienen que distinguirse. Probar justo

en el límite y uno afuera nos dio más confianza que muchos casos en el medio del rango. Todo lo que generó la IA lo revisamos y lo ejecutamos, y podemos explicar por qué está cada caso.

7. Referencias bibliográficas

[1] Universidad ORT Uruguay, Facultad de Ingeniería, "Obligatorio 1 – Diseño de Aplicaciones 2, 2026.2," letra del obligatorio, 2026.

[2] J. Ferreiro, S. Luciardi y A. Bustamante, "M6CD-ID-274754-328344-302420," repositorio en GitHub. [Online]. Available: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420

[3] Postman, "Run collections using the Collection Runner," documentación de Postman. [Online]. Available: https://learning.postman.com/docs/collections/running-collections/intro-to-collection-runs/